



Université de Rouen Normandie

SSL Checker

Dossier d'Architecture Logicielle

Dossier d'Architecture Logicielle

Étudiants :

Mehdi LASMI

Marzouk MONIR

Malik MAHFOUDI

Karina AIT-ABDELKAOUI

Naby TRAORE SAVANE

Rayan BOUGHANEM

Hamza RAHMOUNI

Amr ELGOHARY

Client :

Bruno MACADRÉ

Table des matières

1 Documents de référence **4**

2 Terminologie et sigles utilisés **4**

3 Objet **6**

4 Configuration requise et Choix Technologiques **6**

4.1 Environnement de déploiement 6

4.2 Liste des composants logiciels 6

4.3 Diagramme architecture globale 7

5 Architecture Statique **7**

5.1 Explication générale 7

5.2 Diagramme de classes 8

6 Fonctionnement Dynamique du Modèle **9**

6.1 Liste des Composants du Modèle 9

6.2 Description du Processus de Mise en Œuvre 9

6.2.1 Phase 1 : Instanciation des Modèles (`Factory`) 9

6.2.2 Phase 2 : Réalisation des Tests Techniques 9

6.2.3 Phase 3 : Calcul des Notes 10

6.2.4 Phase 4 : Génération des Rapports 10

6.2.5 Phase 5 : Récupération et Restitution des Résultats 10

6.3 Diagramme de séquence Générale 11

7 Fonctionnement Dynamique de l'Analyse du Certificat **12**

7.1 Composants Mis en Jeu 12

7.2 Description du Processus de Mise en Œuvre 12

7.2.1 Phase 1 : Connexion et Récupération 12

7.2.2 Phase 2 : Analyse Détaillée du Certificat 12

7.2.3 Phase 3 : Restitution 13

7.2.4 Digramme de séquence Analyse Certificat 14

8 Fonctionnement Dynamique du Test des Protocoles TLS/SSL **15**

8.1 Composants Mis en Jeu 15

8.2 Description du Processus de Mise en Œuvre 15

8.2.1 Phase 1 : Initialisation 15

8.2.2 Phase 2 : Test de Supportabilité de Chaque Protocole (EF-07, EF-08) 15

8.2.3 Phase 3 : Retour de la Liste des Protocoles 16

8.2.4 Diagramme de séquence Tests protocole 16

9 Fonctionnement Dynamique du Test des Suites de Chiffrement 16
9.1 Description des Composants Clés 16
9.2 Déroulement de la Séquence TestCipher 17
9.2.1 Phase 1 : Énumération des Suites de Chiffrement (EF-09) 17
9.2.2 Phase 2 : Détermination de l'Ordre de Préférence (EF-21) 17
9.2.3 Diagramme de Séquence Test des Cipher Suites 19

10 Fonctionnement Dynamique de l'Analyse des Algorithmes d'Échange de Clés 20
10.1 Composants Mis en Jeu 20
10.2 Description du Processus de Mise en Œuvre 20
10.2.1 Phase 1 : Initialisation de l'Analyse (EF-25) 20
10.2.2 Phase 2 : Analyse des Algorithmes d'Échange de Clés (EF-26) . . . 20
10.2.3 Phase 3 : Finalisation et Restitution 21
10.2.4 Diagramme de Séquence Échange de Clés 22

11 Fonctionnement Dynamique de l'Analyse des Vulnérabilités 22
11.1 Orchestration Globale des Tests de Vulnérabilités 22
11.1.1 Liste des Composants Mis en Jeu 22
11.1.2 Processus de Mise en Œuvre (Scénario : Vérification des Vulnérabilités) 23
11.1.3 Diagramme de Séquence Generale Tests Vulnerablitite 24
11.2 Focus Sur Test de la Faille Heartbleed 24
11.2.1 Composants Spécifiques 24
11.2.2 Processus (Simulation d'Attaque Active) 24
11.2.3 Diagramme de sequence HeartBleed 25
11.3 Focus Sur Tests de Configuration Forcée 26
11.3.1 Composants Spécifiques 26
11.3.2 Processus (Itératif par Vulnérabilité) 26
11.3.3 Diagramme de séquence Tests de Configuration Forcée 27

12 Fonctionnement Dynamique Test de Compatibilité OS 27
12.1 Liste des Composants Mis en Jeu 27
12.2 Description du Processus de Mise en Œuvre 27
12.2.1 Phase 1 : Récupération et Préparation des Données 28
12.2.2 Phase 2 : Algorithme de Vérification (Boucle de Compatibilité) . . 28
12.2.3 Phase 3 : Sortie Finale (Structure de Données) 29
12.2.4 Diagramme de séquence Compatibilité OS 29

13 Fonctionnement Dynamique du Scan de Ports **30**
13.1 Liste des Composants Mis en Jeu 30
13.2 Description du Processus de Mise en Œuvre 30
13.2.1 Phase 1 : Lancement et Préparation 30
13.2.2 Phase 2 : Phase Technique – Scan TCP SYN 30
13.2.3 Phase 3 : Transmission des Résultats 31
13.2.4 Diagramme de Séquence Scan de Ports 32

14 Note et Rapport **32**
14.1 Calcul de la Note Globale de Sécurité 32
14.1.1 Règle de Conditionnement Initiale (Certificat) 33
14.1.2 Règles de Scoring Pondéré par Catégorie Technique 33
14.1.3 Formule de Calcul et Conversion du Grade 35
14.2 Structuration du Résultat Final 36
14.2.1 Rôles des Composants dans la Restitution 36
14.2.2 Structure de l'Objet de Sortie JSON 36

15 Fonctionnement du Frontend **37**
15.1 Liste des Composants de l'Interface Utilisateur 37
15.2 Description du Processus de Mise en Œuvre 38
15.2.1 Phase 1 : Saisie et Requête Asynchrone 38
15.2.2 Phase 2 : Réception de la Réponse 38
15.2.3 Phase 3 : Affichage Des Blocs 38

16 Traçabilité **39**
16.1 Fonctionnelles 39
16.2 Non-Fonctionnnelles 41

1 Documents de référence

— STB : Définit les besoins, les exigences fonctionnel et non fonctionnel.

2 Terminologie et sigles utilisés

— **API (Application Programming Interface)** : Interface permettant au Frontend (JavaScript) de communiquer avec le Backend (Python) pour envoyer des requêtes et recevoir des données.
— **Backend** : Partie "serveur" de l'application (ici en Python/Flask) qui gère la logique métier, les tests , la génération de rapport et le calcul des notes.
— **CRL (Certificate Revocation List)** : Liste de certificats révoqués par l'autorité de certification avant leur date d'expiration.
— **DER (Distinguished Encoding Rules)** : Format d'encodage binaire utilisé pour stocker les certificats X.509.
— **Docker** : Plateforme de conteneurisation permettant de déployer l'application et ses dépendances (Python, Nginx, etc.) de manière identique sur n'importe quel environnement.
— **DOM (Document Object Model)** : Structure arborescente représentant la page web, manipulée par le JavaScript pour afficher les résultats dynamiquement.
— **Flask** : Micro-framework web écrit en Python, utilisé ici pour créer l'API et servir l'application.
— **Frontend** : Partie "client" de l'application (HTML/CSS/JS) qui s'exécute dans le navigateur de l'utilisateur.
— **FQDN (Fully Qualified Domain Name)** : Nom de domaine complet identifiant une machine (ex : `admin.tryhackme.com`).
— **Gunicorn** : Serveur HTTP WSGI utilisé pour exécuter l'application Python Flask en production.
— **Handshake** : Processus de négociation initial SSL/TLS permettant au client et au serveur de s'accorder sur les clés et les algorithmes de chiffrement.
— **HSTS (HTTP Strict Transport Security)** : Mécanisme de sécurité web forçant les navigateurs à n'interagir avec le serveur qu'en HTTPS sécurisé.
— **JSON (JavaScript Object Notation)** : Format standard d'échange de données textuelles, utilisé pour structurer la réponse renvoyée par le Contrôleur.
— **Nginx** : Serveur web haute performance utilisé comme proxy inverse pour gérer les connexions entrantes et les fichiers statiques.
— **OCSP (Online Certificate Status Protocol)** : Protocole permettant de vérifier en temps réel l'état de validité d'un certificat SSL.
— **OpenSSL** : Bibliothèque cryptographique robuste utilisée par le Backend pour

effectuer les connexions SSL/TLS et analyser les protocoles.

— **SYN / ACK / RST** : Drapeaux (Flags) du protocole TCP utilisés lors du scan de ports pour déterminer si un port est ouvert, fermé ou filtré.

— **Socket** : Point de terminaison d'une connexion réseau bidirectionnelle, utilisé ici pour les tests bas niveau (Raw Sockets).

— **X.509** : Standard cryptographique définissant le format des certificats à clé publique.

3 Objet

L'objectif de ce document est de décrire l'architecture logicielle retenue pour le projet **SSL Checker**. Il présente la solution technique conçue pour répondre aux exigences définies dans la Spécification Technique des Besoins (STB).

Ce document identifie les différents modules du système ainsi que leurs interfaces, afin de guider le développement et l'intégration des composants par l'équipe projet.

Rappel du besoin

Concevoir et réaliser un outil d'audit de sécurité (**SSL Checker**) fonctionnant en intranet. L'outil doit proposer une interface web simple, s'inspirer du service *SSL Labs* de Qualys, et fournir une analyse complète de la configuration SSL/TLS d'un serveur cible.

4 Configuration requise et Choix Technologiques

Nous étions libres de choisir les technologies pour ce projet. La seule contrainte imposée est l'hébergement : l'application finale doit tourner sur une Machine Virtuelle fournie par l'université.

4.1 Environnement de déploiement

L'application est conçue pour fonctionner sur un environnement Linux standard. Pour éviter les problèmes de compatibilité lors de l'installation sur la VM du client, nous utilisons **Docker**. Cela permet d'avoir un environnement identique entre nos machines de développement et le serveur de production, éviter les : "*ça marchait bien chez moi*"

4.2 Liste des composants logiciels

Nous avons privilégié des outils simples et robustes, adaptés au traitement réseau :

- **Backend (Logique) : Python 3 avec Flask.** C'est le choix le plus logique pour ce projet car Python possède d'excellentes librairies pour analyser le SSL/TLS nativement. Flask est utilisé pour créer l'interface web car il est très léger.
- **Frontend (Interface) : HTML, CSS et JavaScript.** L'interface étant simple, nous n'avons pas besoin de framework lourd (type React ou Angular). Nous utilisons les langages standards du web.
- **Serveur Web : Nginx et Gunicorn.** En production, l'application Flask ne tourne pas seule. Nous utilisons **Gunicorn** pour exécuter le code Python, placé derrière **Nginx** qui gère les connexions entrantes et sert les fichiers statiques.

4.3 Diagramme architecture globale

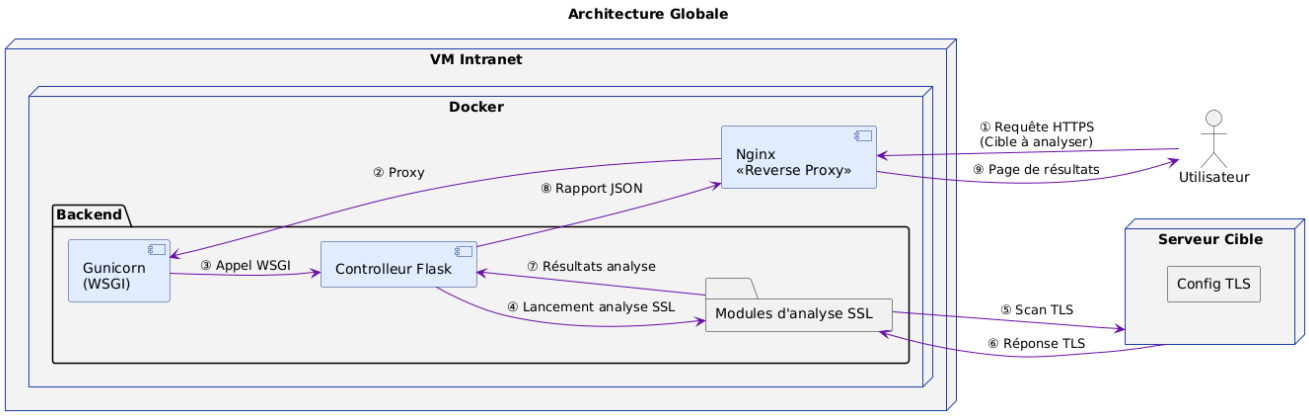


Diagramme architecture globale

5 Architecture Statique

5.1 Explication générale

L'architecture repose sur un motif d'agrégation. Pour structurer les résultats, l'application utilise une hiérarchie de classes :

- **Classes Globales (Conteneurs) :**
  - `SslChecker (SC)` : Objet racine qui prend le FQDN et éventuellement le port lors de sa création. Il instancie Factory (F),puis lance la logique métier, et permet au contrôleur de récupérer la réponse.
  - `Factory (F)` : Objet qui sert uniquement à la création des objets RG, NG et TG.
  - `RapportGlobal (RG)` : Objet qui contient l'intégralité des sous rapports de chaque partie de l'audit .
  - `NoteGlobale (NG)` : Objet chargé de calculer la note finale en pondérant les sous-notes.
  - `TestGlobal (TG)` : Objet qui permet d'effectuer l'ensemble des tests sur le serveur cible .
- **Classes Spécifiques (Composants) :** Chaque axe d'analyse possède ses propres classes pour les trois axes : Test, Note et Rapport . Les classes de la forme RapportX, NoteX, et TestX sont respectivement des sous classes des classes mères : RapportGlobal, NoteGlobale et TestGlobal :
  - `RapportProtocole`, `NoteProtocole` et `TestProtocole` (pour TLS/SSL).
  - `RapportCipher`, `NoteCipher` et `TestCipher` (pour les suites cryptographiques).
  - `RapportCertificat`, `NoteCertificat` et `TestCertificat`.

— `TestEchangeCle`, `NoteEchangeCle` et `RapportEchangeCle`,.

— `RapportVulnerabilites`, `TestVulnerabilites`.

— `RapportPorts`, `TestPorts`.

— `RapportCompatibiliteOs`.

Clarification

On devine assez facilement les rôles des sous classes vu leurs noms et les explications des rôles de leurs classes mères.

5.2 Diagramme de classes

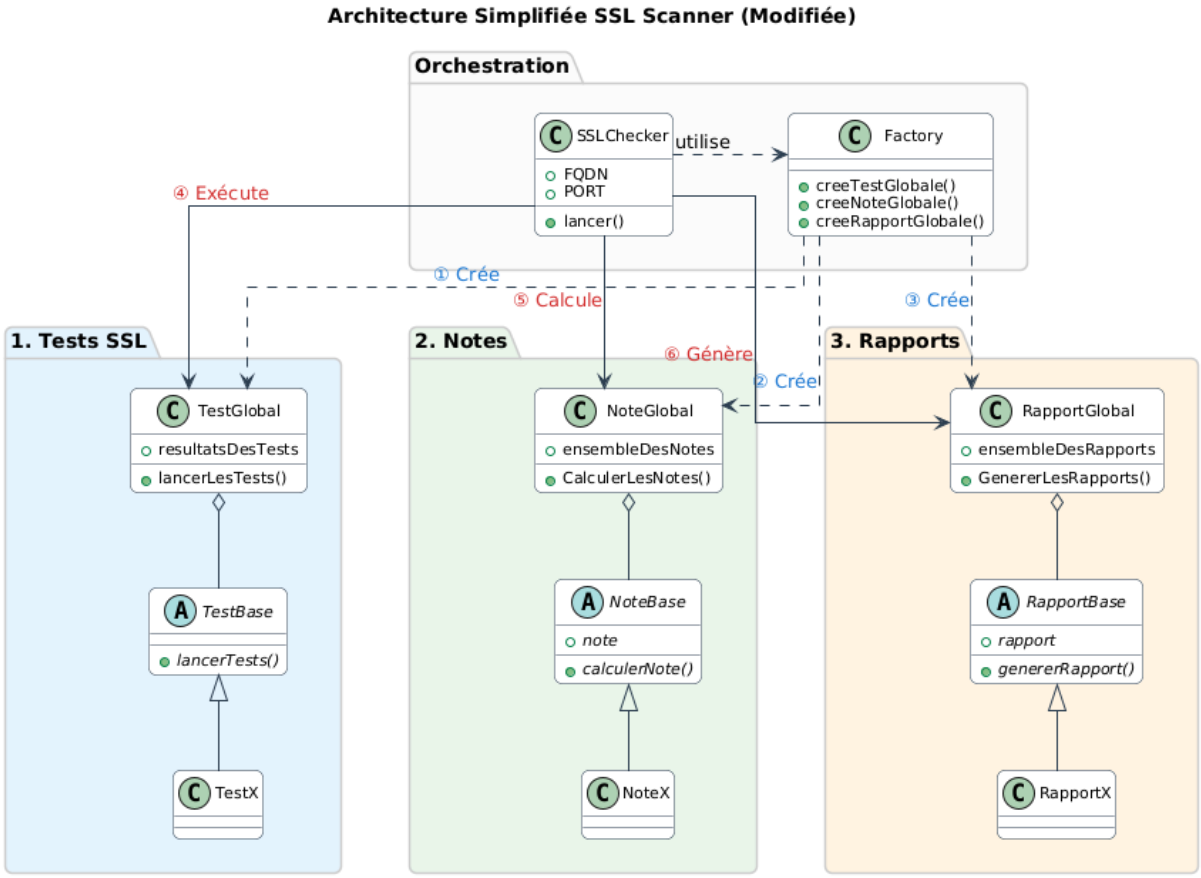


Diagramme de Classes

6 Fonctionnement Dynamique du Modèle

6.1 Liste des Composants du Modèle

Les composants du système sont classés par leur rôle dans l'architecture :

Utilisateur Acteur principal qui saisit le FQDN de la cible et déclenche l'analyse via l'interface web.

Vue Composant chargé de récupérer les actions de l'utilisateur et d'afficher le résultat final (rapport et note).

Contrôleur Point d'entrée applicatif. Il reçoit la demande de la `Vue`, déclenche l'analyse via `SslChecker` et renvoie le résultat.

SslChecker , **Factory**, **NoteGlobale**, **RapportGlobal**, **TestGlobal**.

NoteX , RapportX, TestX .

6.2 Description du Processus de Mise en Œuvre

Le processus d'analyse se déroule en cinq phases séquentielles :

6.2.1 Phase 1 : Instanciation des Modèles (`Factory`)

1. L'**Utilisateur** saisit le FQDN. La `Vue` transmet la requête `lancerAnalyse(FQDN)` au `Contrôleur`.
2. Le `Contrôleur` crée un objet `SSLChecker`.
3. `SSLChecker` instancie la `Factory`, qui crée les principaux composants : **NoteGlobale**, **RapportGlobal** et **TestGlobal**.
4. **NoteGlobal** crée les sous-notes nécessaires (représentées par `NoteX`).
5. **RapportGlobal** crée les sous-rapports spécialisés (représentés par `RapportX`).
6. **TestGlobal** instancie les modules de tests techniques (représentés par `TestX`).
7. **Factory** retourne à `SSLChecker` les références vers les trois objets globaux.

6.2.2 Phase 2 : Réalisation des Tests Techniques

— `SSLChecker` appelle `lancerLesTests()` sur `TestGlobal`.
— `TestGlobal` pilote l'exécution des différents tests spécialisés regroupés dans `TestX`.
— Chaque module de test exécute ses vérifications sur la cible et ajoute ses résultats bruts dans une structure partagée : **listeResultats**.

À savoir !

À l'issue de cette phase, `listeResultats` contient toutes les données techniques nécessaires.

6.2.3 Phase 3 : Calcul des Notes

- `SSLChecker` déclenche le calcul des notes en appelant `calculerLesNotes(ListeRésultats)` sur **NoteGlobale**.
- **NoteGlobale** délègue ce travail aux notes spécialisées (`NoteX`).
- Chaque composant `NoteX` lit la partie de `ListeRésultats` qui le concerne et calcule une **sous-note locale**.
- Ces sous-notes sont conservées au niveau de `NoteGlobale` (ex : `noteC`, `noteP`, `noteCS` et `noteEC`)

6.2.4 Phase 4 : Génération des Rapports

- En parallèle, `SSLChecker` demande à **RapportGlobal** de générer les rapports via `genererLesRapports(ListeRésultats)`.
- **RapportGlobal** fait appel aux différents rapports spécialisés (`RapportX`).
- Chaque module `RapportX` produit un **rapport détaillé** à partir des données de `ListeRésultats`.
- Ces sous-rapports sont conservés dans `RapportGlobal`.

6.2.5 Phase 5 : Récupération et Restitution des Résultats

- `SSLChecker` demande à **NoteGlobale** de calculer la note globale ...
- `SSLChecker` récupère cette note ainsi que les sous-notes et le rapport auprès de `RapportGlobal`.
- Le résultat (Notes et Rapport) est renvoyé par le `Contrôleur` à la `Vue` pour affichage à l'`Utilisateur`.

6.3 Diagramme de séquence Générale

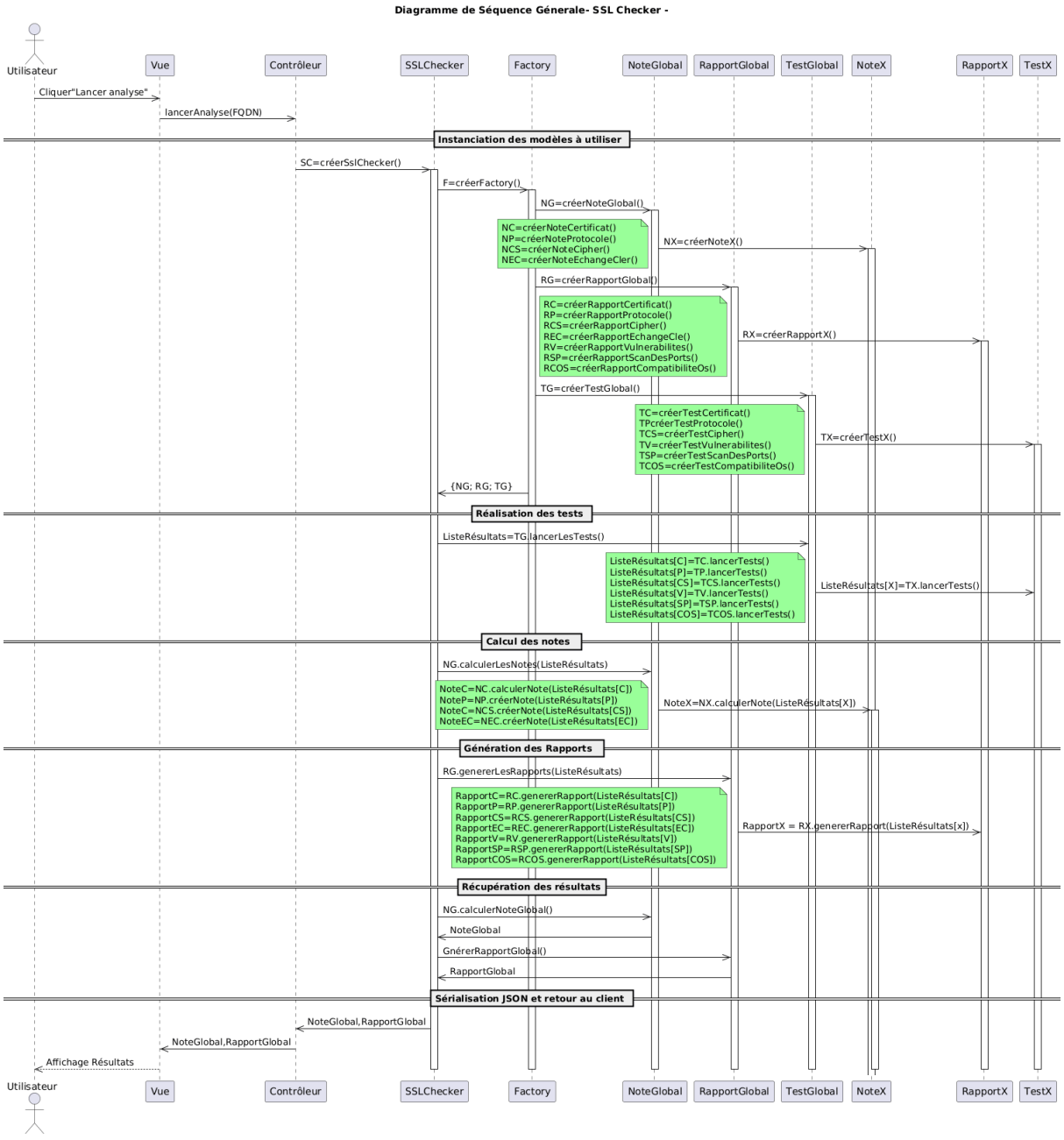


Diagramme de séquence général

7 Fonctionnement Dynamique de l'Analyse du Certificat

7.1 Composants Mis en Jeu

Le moteur d'analyse s'articule autour des composants suivants :

— **TestGlobal**.

— **TestCertificat** : Classe spécialisée chargée de piloter la récupération et l'analyse spécifique du certificat **X.509**.

— **Serveur Cible** : Le serveur web (identifié par FQDN) dont on analyse la configuration SSL/TLS.

— **Cryptography** : Bibliothèque technique (`OpenSSL`/`Cryptography` en Python) utilisée pour parser les données binaires (**DER**) en objets manipulables en Python.

7.2 Description du Processus de Mise en Œuvre

Ce processus suit le flux d'événements du scénario "Analyse du Certificat", divisé en trois phases.

7.2.1 Phase 1 : Connexion et Récupération

1. **Initiation** : `TestGlobal` demande à `TestCertificat` de lancer l'analyse complète pour un domaine donné.
2. **Handshake Réseau** :
   — `TestCertificat` ouvre une **socket TCP** vers le **Serveur Cible** sur le port `443`.
   — Il initie un **handshake SSL/TLS** (`wrap_socket`) pour récupérer le certificat présenté par le serveur sans envoyer de requête HTTP.
3. **Parsing** : Une fois les données binaires reçues, `TestCertificat` utilise la bibliothèque `Cryptography` pour convertir le format binaire (**DER**) en un objet certificat structuré (**X.509**).

7.2.2 Phase 2 : Analyse Détaillée du Certificat

Une fois l'objet certificat chargé, `TestCertificat` l'analyse finement pour vérifier chaque aspect critique requis par la STB.

— **Analyse de l'Autorité (EF-05)** :
   — La méthode `analyseAutorite()` vérifie si le certificat est **auto-signé** ou émis par une autorité tierce.

— **Analyse de Validité (EF-06)** :
  — La méthode `analyseValidite()` compare les dates `notBefore` et `notAfter` avec la date actuelle.
  — Elle calcule les jours restants avant expiration.
— **Analyse Clé Publique et Signature** (`analyseClePublique()` & `analyseSignature()`) :
  — Vérification de la **robustesse de la clé** (ex : RSA 2048 bits minimum) et de l'algorithme de signature (ex : rejet du SHA-1 au profit de SHA-256).
— **Analyse des Extensions** (`analyseExtensions()`) :
  — Extraction des **SAN** (Subject Alternative Names) pour vérifier que le domaine est couvert.
  — Vérification des **usages de clé (KeyUsage)** et des **Extended Key Usage (EKU)** pour l'authentification serveur.
— **Analyse de la Chaîne de Certification (RG-07)** :
  — La méthode `analyseChaineCertification()` reconstitue la chaîne jusqu'à une racine (**Root CA**) et valide les signatures intermédiaires.
— **Analyse de Révocation** (`analyseRevocation()`) :
  — Le système interroge les répondeurs **OCSP** ou les listes **CRL** pour s'assurer que le certificat n'a pas été révoqué.

7.2.3 Phase 3 : Restitution

— `TestCertificat` agrège tous les résultats partiels.
— Il retourne un résultat à `TestGlobal`.

7.2.4 Digramme de séquence Analyse Certificat

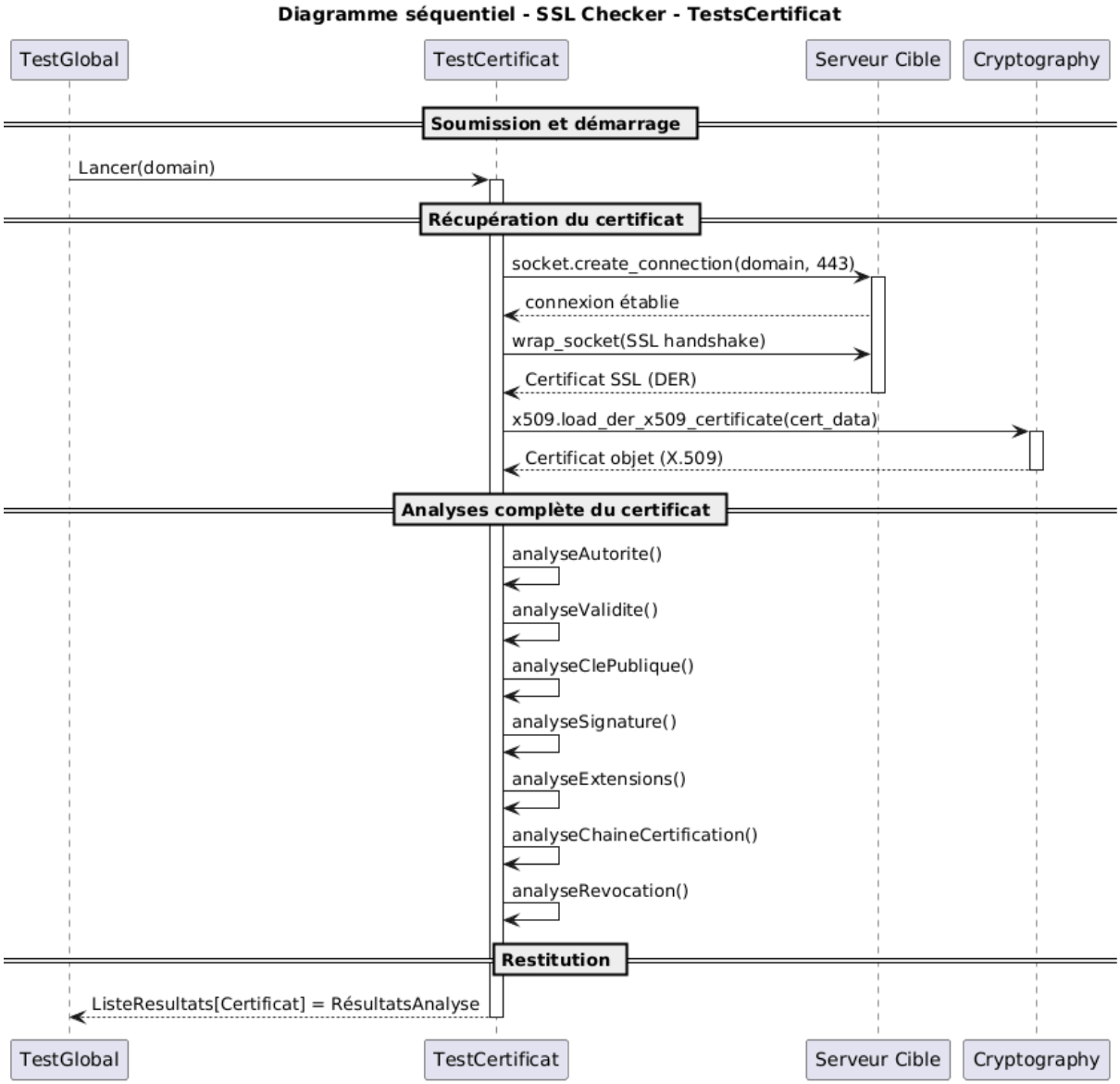


Diagramme séquence Analyse Certificat

8 Fonctionnement Dynamique du Test des Protocoles TLS/SSL

8.1 Composants Mis en Jeu

L'analyse du support des protocoles repose sur deux composants principaux :

— **TestGlobal**.

— **TestProtocole** : Classe spécialisée chargée de piloter les tests de supportabilité pour l'ensemble des protocoles : **TLS 1.3**, **TLS 1.2**, **TLS 1.1**, **TLS 1.0**, **SSL 3.0** et **SSL 2.0**.

8.2 Description du Processus de Mise en Œuvre

Ce processus suit le flux d'événements du scénario "Tester les protocoles" et s'exécute en trois phases.

8.2.1 Phase 1 : Initialisation

1. `TestGlobal` instancie un objet de la classe `TestProtocole`.
2. La fonction `lancer()` de cet objet est appelée en fournissant, au minimum, le **FQDN** (nom de domaine) de la cible sous forme de chaîne de caractères.

8.2.2 Phase 2 : Test de Supportabilité de Chaque Protocole (EF-07, EF-08)

Cette phase est une boucle itérant de `TLS 1.3` jusqu'à `SSL 2.0`.

1. **Construction de la Commande** : Une commande `OpenSSL` est construite pour forcer une tentative de **handshake** en utilisant uniquement le protocole ciblé (ex : `openssl s_client -tls1_2 ...`).
2. **Exécution** : La commande est exécutée via la fonction `subprocess.run()` (ou équivalent), qui retourne un objet de type `CompletedProcess`.
3. **Analyse du Résultat** :
   — Le processus vérifie l'attribut `stdout` de l'objet retourné (de type `str`).
   — La présence de la chaîne de caractères `"CONNECTED"` indique que le protocole est **supporté** (`True`). Son absence indique qu'il est **non supporté** (`False`).
4. **Mise à Jour des Résultats** : La liste des résultats est mise à jour avec le statut du protocole testé.

```
{ "protocole": "TLS 1.2", "supporte": True, "obsolete": False }
```

8.2.3 Phase 3 : Retour de la Liste des Protocoles

— La fonction `lancer()` renvoie la **liste complète des résultats** pour tous les protocoles testés.

— Cette liste inclut, pour chaque protocole, les champs `protocole`, `supporte` (booléen) et `obsolète` (booléen).

8.2.4 Diagramme de séquence Tests protocole

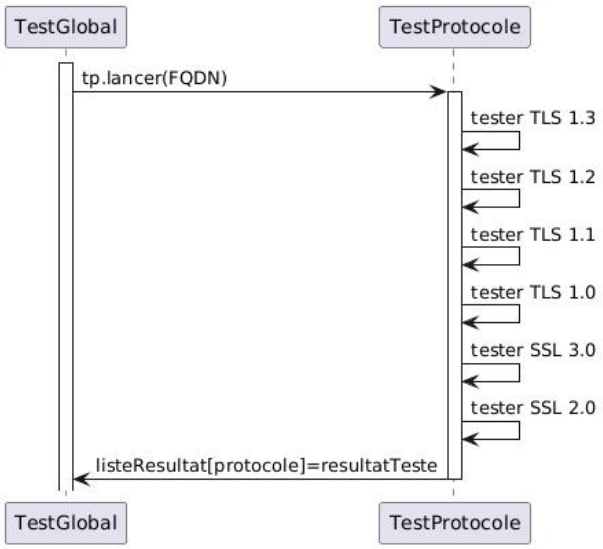


Diagramme de séquence Tests protocole

9 Fonctionnement Dynamique du Test des Suites de Chiffrement

9.1 Description des Composants Clés

L'exécution du test des suites de chiffrement (`ciphertest`) implique les composants suivants :

TestGlobal, Serveur Cible .

TestCipher Classe d'implémentation de la logique de test. Elle exécute la logique de vérification, récupère les résultats, et prépare les données (`resultatcipher`) pour les modules de notation et de rapport. Elle interagit directement avec le **Serveur Cible** via l'envoi de requêtes `ClientHello`.

CipherData Source de données contenant la liste exhaustive des suites de chiffrement de référence à tester. `TestCipher` y accède pour récupérer la liste complète des `Ciphersuites`.

9.2 Déroulement de la Séquence TestCipher

Le processus d'analyse des suites de chiffrement est divisé en deux phases distinctes.

9.2.1 Phase 1 : Énumération des Suites de Chiffrement (EF-09)

Initialisation

1. La classe `TestGlobal` demande à l'objet `TestCipher` d'exécute sa méthode `lancer()`.
2. Elle passe en paramètre la partie "protocole" de l'objet `listeResultats`.
3. `TestCipher` charge la liste des ciphers de référence depuis un fichier et initialise une liste de sortie.

Exécution de la Boucle de Test (Protocole × Cipher) Le programme parcourt la liste des protocoles supportés et, pour chaque protocole, itère sur l'ensemble des suites de chiffrement de référence.

Processus de Communication (Handshake) À chaque itération, un message `ClientHello` est envoyé au serveur :

— **Succès** (`ServerHello`) : La suite de chiffrement est acceptée. Elle est ajoutée à la liste `resultatCipher` (Ordre de préférence initialisé à 0).

— **Échec** (Message d'alerte) : La combinaison n'est pas supportée. Le programme passe à l'itération suivante.

— **Exception** (Pas de réponse) : Une exception est générée.

9.2.2 Phase 2 : Détermination de l'Ordre de Préférence (EF-21)

Vérification de la Préférence du Serveur Cette étape vérifie si le serveur impose un **Cipher Order** strict.

1. **Test Initial** : `TestCipher` sélectionne deux suites de chiffrement (A et B) validées en Phase 1.
2. **Requêtes** : Deux `ClientHello` sont envoyés : un avec l'ordre [A, B] et un avec l'ordre inversé [B, A].
3. **Analyse des Réponses** :
   — **Réponses Différentes (A puis B)** : Le serveur respecte l'ordre du client. Il n'a pas de préférence stricte. On passe à l'étape finale.
   — **Réponses Identiques (A puis A)** : Le serveur impose sa préférence. L'algorithme de classement doit être exécuté.

Détermination des Préférences (Ranking) Si une préférence serveur est détectée, l'algorithme suivant est exécuté pour classer les ciphers :

1. **Initialisation** : Une copie temporaire de `resultatCipher` est générée.
2. **Boucle de Tri** : Tant que la liste temporaire n'est pas vide :
   — Un `ClientHello` est envoyé avec toutes les suites restantes de la liste temporaire.
   — Le serveur choisit sa suite préférée.
   — L'ordre de cette suite dans `resultatCipher` est mis à jour (1er, 2e, etc.).
   — La suite est retirée de la liste temporaire.
3. **Finalisation** : `TestCipher` retourne la liste `resultatCipher` complétée à `TestGlobal`.

9.2.3 Diagramme de Séquence Test des Cipher Suites

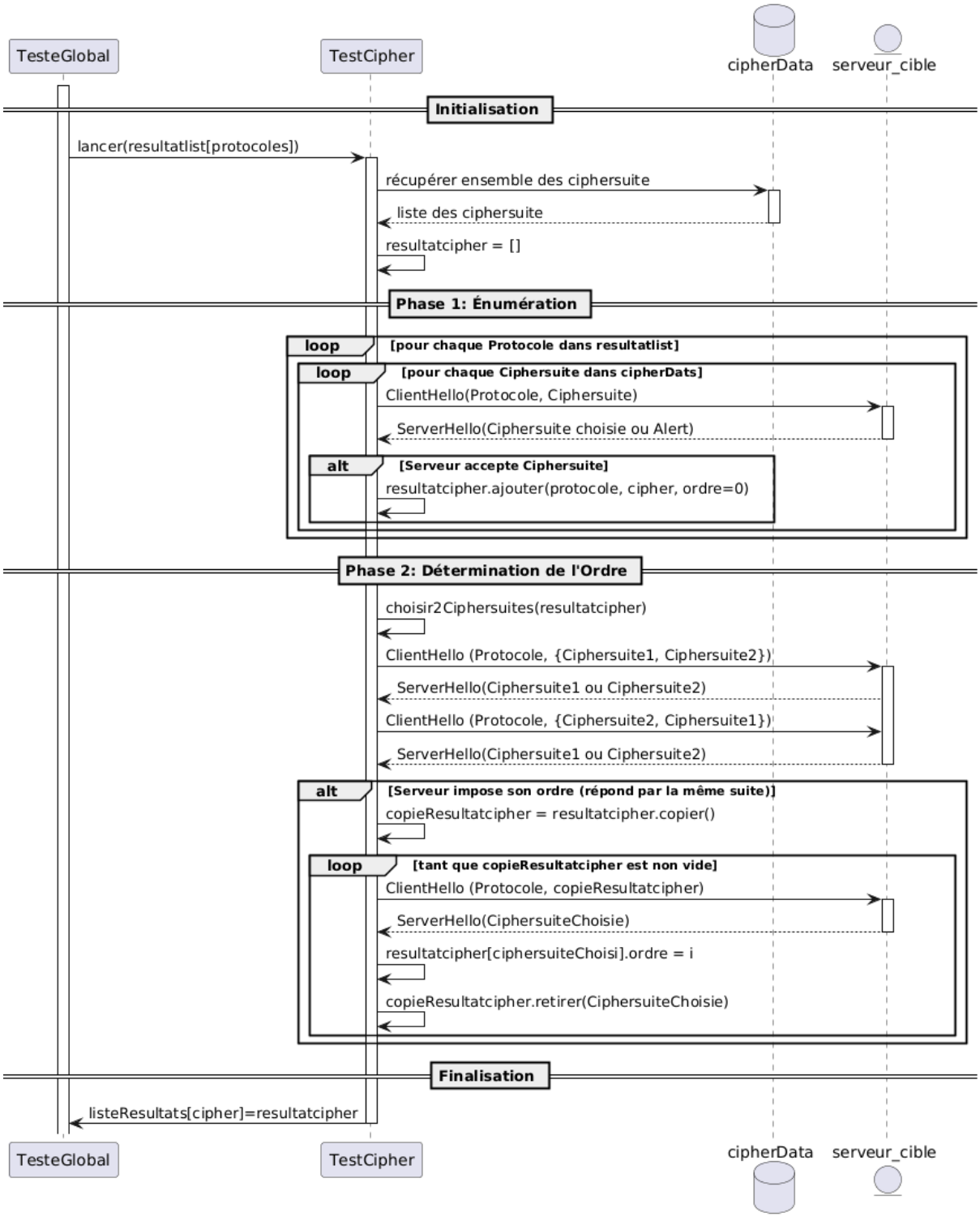


Diagramme de Séquence Test des Cipher Suites

10 Fonctionnement Dynamique de l'Analyse des Algorithmes d'Échange de Clés

10.1 Composants Mis en Jeu

L'analyse des mécanismes d'échange de clés repose sur les composants suivants :

— **TestGlobal, Serveur**

— **TestEchangeCle** : Composant spécialisé chargé d'analyser les mécanismes d'échange de clés. Il identifie les algorithmes supportés par le serveur ainsi que la taille des clés associées.

10.2 Description du Processus de Mise en Œuvre

Ce processus suit le flux d'événements du scénario "Analyse de l'Échange de Clés", divisé en trois phases.

10.2.1 Phase 1 : Initialisation de l'Analyse (EF-25)

1. **Appel** : `TestGlobal` démarre le processus en appelant la méthode `lancer()` du module `TestEchangeCle`.
2. **Données d'entrée** : La méthode prend en entrée :
   — La liste des suites de chiffrement déjà identifiées (`listeResultats[cipher]`).
   — Les informations du certificat de la cible (`listeResultats[certificat]`).
3. **Préparation** :
   — `TestEchangeCle` initialise une liste vide (`kexResultats`) pour stocker les résultats.
   — Il regroupe les suites de chiffrement par type d'échange via la méthode `grouperKex()`.

10.2.2 Phase 2 : Analyse des Algorithmes d'Échange de Clés (EF-26)

`TestEchangeCle` entre dans une boucle pour analyser chaque groupe de suites. Le traitement se divise en deux branches selon la nature de l'algorithme.

Branche A : Algorithmes Statiques (RSA, DH statique) Pour ces mécanismes, aucune communication réseau supplémentaire n'est nécessaire.

— La clé publique se trouve déjà dans le certificat.

— `TestEchangeCle` récupère les paramètres via `certificat.params_clé_publique`.

— Il calcule la taille de la clé via la méthode `analyseClé()`.

Branche B : Algorithmes Éphémères (DHE, ECDHE) Pour les mécanismes utilisant un échange éphémère, `TestEchangeCle` interroge le serveur via un échange TLS simplifié.

1. **Envoi** : Un message `ClientHello` est envoyé, précisant le protocole et la suite à tester.
2. **Réponse** : Le serveur répond avec un `ServerHello`.
3. **Extraction des Paramètres** :
   — *Protocole* $\neq$ *TLS 1.3* : Le serveur envoie un `ServerKeyExchange`. `TestEchangeCle` analyse les paramètres (Diffie-Hellman ou Courbe Elliptique) pour déduire la taille.
   — *Protocole* = *TLS 1.3* : Les paramètres se trouvent dans l'extension `server_share`. `TestEchangeCle` lit cette extension pour extraire la taille.

À la fin de chaque itération, le couple (Algorithme, Taille) est ajouté à la liste `kexResultats`.

10.2.3 Phase 3 : Finalisation et Restitution

— Une fois l'ensemble des algorithmes analysés, `TestEchangeCle` retourne la liste complète `kexResultats` à `TestGlobal`.
— `TestGlobal` met à jour la structure principale des résultats via `listeResultats[KeyExchange]`.

10.2.4 Diagramme de Séquence Échange de Clés

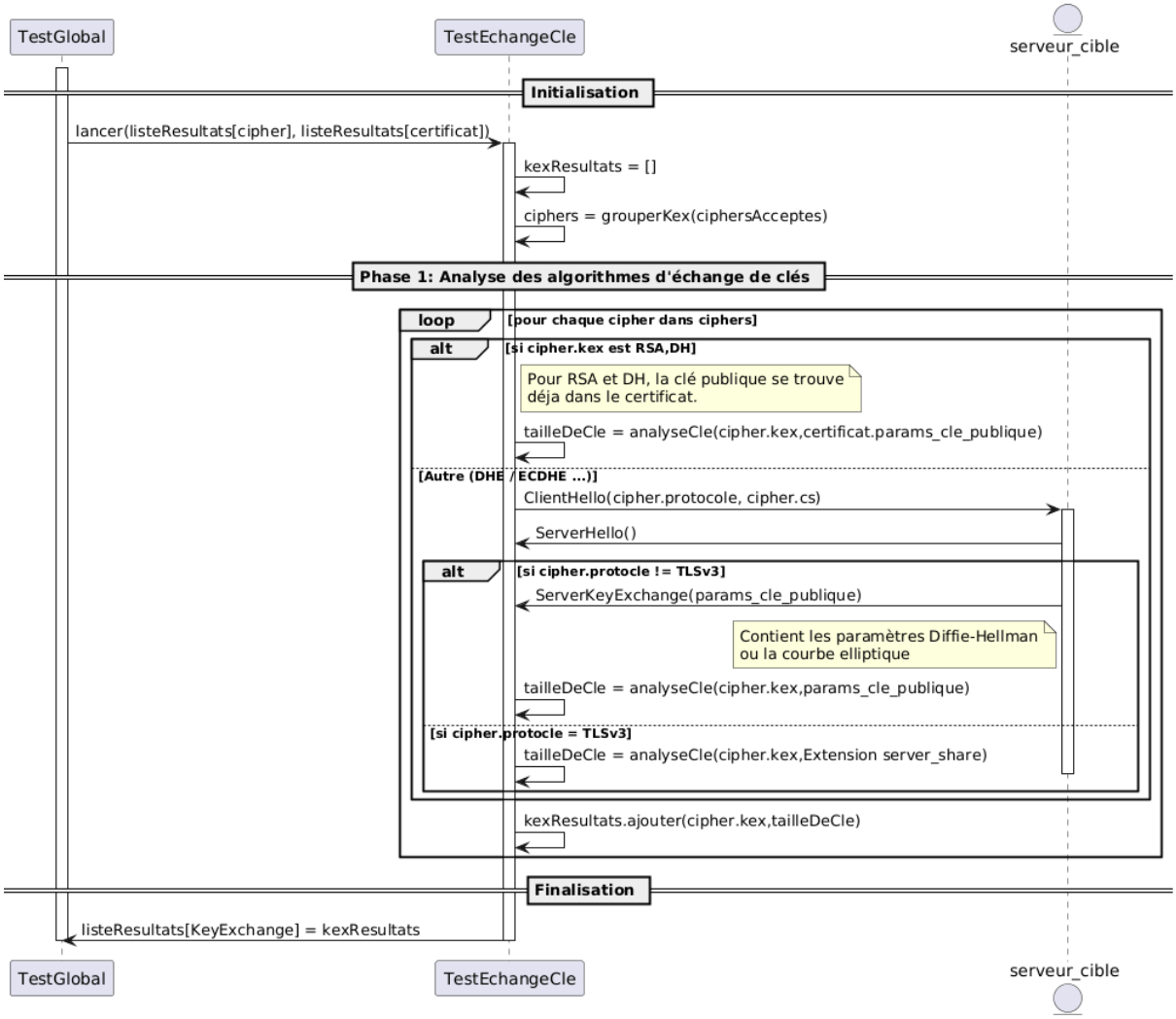


Diagramme de Séquence Analyse Échange de Clés

11 Fonctionnement Dynamique de l'Analyse des Vulnérabilités

11.1 Orchestration Globale des Tests de Vulnérabilités

11.1.1 Liste des Composants Mis en Jeu

TestsGlobale .

TestVulnerabilites Classe de contrôle chargée d'orchestrer l'ensemble des simulations d'attaques et de vérifier la robustesse du serveur.

TestVulnConfig Classe spécialisée testant les vulnérabilités liées aux mauvaises configurations protocolaires (POODLE, SWEET32, etc.).

TestHeartbleed Classe spécialisée testant la faille d'implémentation Heartbleed.

TestHSTS Classe spécialisée vérifiant les mécanismes de durcissement (*Hardening*) via les en-têtes HTTP.

11.1.2 Processus de Mise en Œuvre (Scénario : Vérification des Vulnérabilités)

Phase 1 : Initiation et Orchestration

1. **Appel Principal** : `TestsGlobale` demande à `TestVulnerabilites` de lancer l'analyse de sécurité (`LancerTestsVulnerabilites`).
2. **Initialisation** : `TestVulnerabilites` initialise une liste vide `ListResultatVulnerabilites` pour agréger les résultats.

Phase 2 : Exécution des Simulations

— **Tests de Configuration (EF-12)** : Délégation à `TestVulnConfig` (détail en section 9.3).

— **Tests d'Implémentation (EF-11)** : Délégation à `TestHeartbleed` (détail en section 9.2).

Phase 3 : Analyse du Durcissement (HSTS)

1. **Lancement** : Appel direct de `LancerTestHSTS` avec la connexion TLS active.
2. **Analyse** : Envoi d'une requête HTTPS standard et lecture des en-têtes.
3. **Interprétation** :
   — Si la clé `Strict-Transport-Security` est **absente** : Serveur vulnérable (→ `"HSTS": True`).
   — Si la clé est **présente** : Serveur sécurisé (→ `"HSTS": False`).

Phase 4 : Restitution

— Renvoi dela liste `ListResultatVulnerabilites` à `TestGlobal`.

11.1.3 Diagramme de Séquence Generale Tests Vulnerablitite

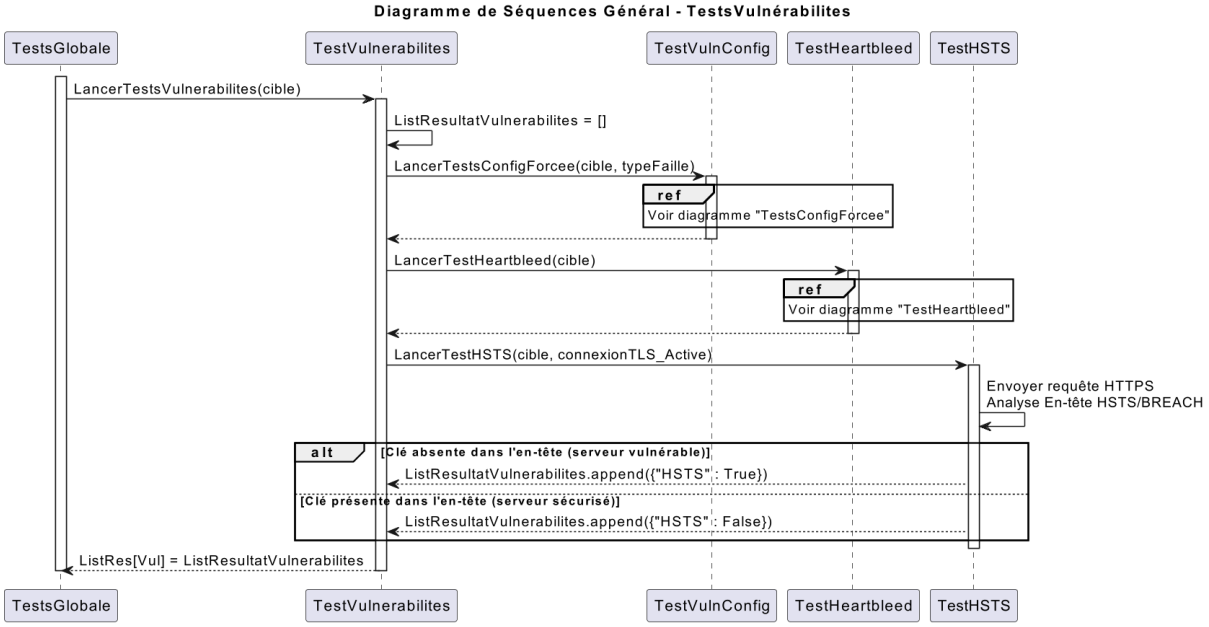


Diagramme de Sequence Generale Tests Vulnerablitite

11.2 Focus Sur Test de la Faille Heartbleed

11.2.1 Composants Spécifiques

- **TestsVulnerabilites** : Contrôleur déclencheur.
- **TestHeartbleed** : Expert chargé de construire le paquet malformé.
- **Bibl :SocketRaw** : Bibliothèque bas niveau permettant d'ouvrir des sockets TCP bruts et d'envoyer des octets sans la surcouche de protection SSL standard de Python.

11.2.2 Processus (Simulation d'Attaque Active)

Phase 1 : Connexion Bas Niveau

- `TestHeartbleed` utilise `Bibl:SocketRaw` pour ouvrir une connexion TCP directe (`ouvrirConnexionTCP`) vers le port cible, contournant volontairement le module SSL sécurisé.

Phase 2 : Injection (Flux Logique Interne)

1. **Construction** : Génération manuelle d'un paquet binaire malformé (`construirePaquetHeartbleed()`).

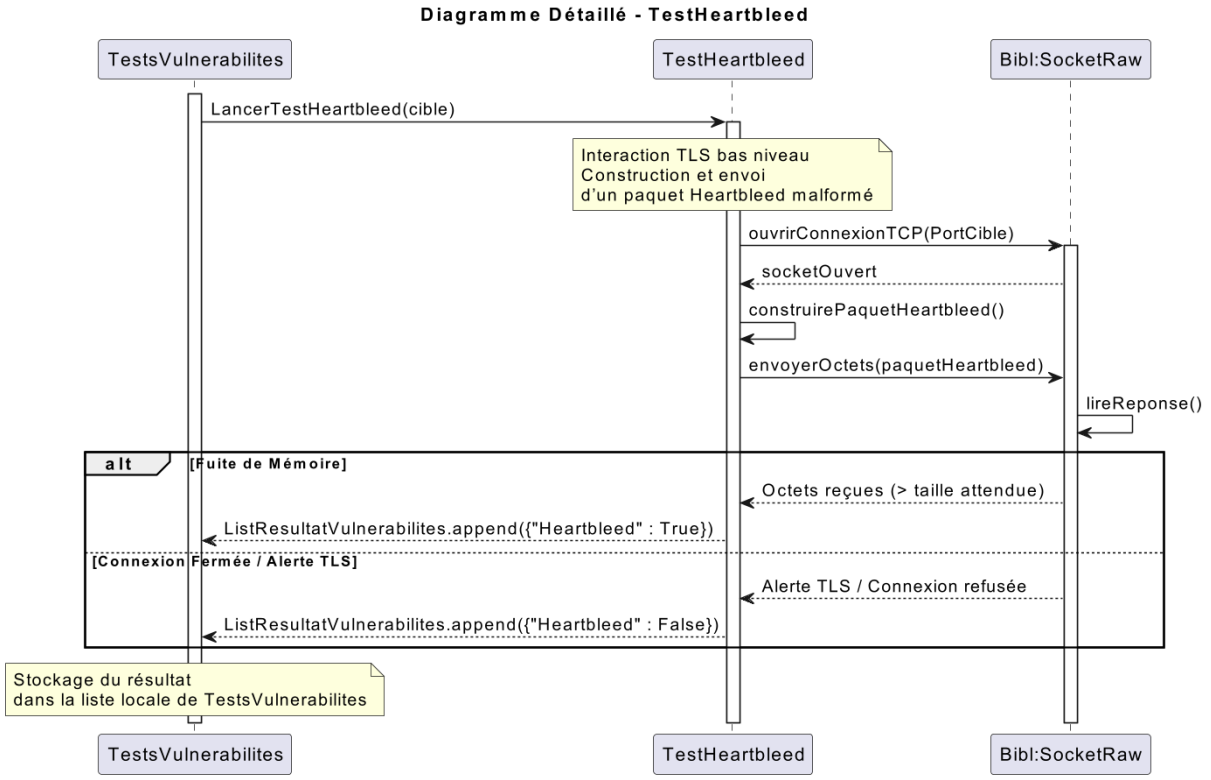
2. **Payload Falsifié** : Le paquet annonce une taille de payload (ex : 64KB) bien supérieure à la taille réelle des données envoyées.
3. **Envoi** : Injection des octets bruts vers le serveur.

Phase 3 : Analyse de la Réponse Lecture de la réponse via `lireReponse()`.

— **Scénario Vulnérable (Fuite de Mémoire)** :
  — Le serveur renvoie un nombre d'octets correspondant à la taille falsifiée (données de sa RAM).
  — Résultat : `"Heartbleed": True`.

— **Scénario Sécurisé** :
  — Le serveur ne répond pas, ferme la connexion ou renvoie une alerte TLS.
  — Résultat : `"Heartbleed": False`.

11.2.3 Diagramme de sequence HeartBleed



11.3 Focus Sur Tests de Configuration Forcée

Concerne les failles protocolaires type POODLE, SWEET32, etc.

11.3.1 Composants Spécifiques

— **TestsConfigForcée** : Exécute un handshake TLS avec des paramètres dégradés.
— **Bibl : TLSContextBuilder** : Bibliothèque utilitaire pour fabriquer des contextes SSL/TLS personnalisés (forcer une version ou un cipher).

11.3.2 Processus (Itératif par Vulnérabilité)

Phase 1 : Itération et Préparation

1. **Boucle** : `TestsVulnerabilites` itère sur la liste des failles connues.
2. **Création de Contexte** : Appel à `creerContexte(type_faille)`.
3. **Configuration Toxique** : L'outil génère un contexte vulnérable (ex : forcer SSLv3, forcer l'algorithme 3DES, ou activer la compression TLS).

Phase 2 : Simulation de Handshake

— Tentative d'initiation d'un `ClientHello` vers le Serveur Cible en utilisant exclusivement le **contexteVulnerable**.

Phase 3 : Interprétation du Résultat

— **Scénario Vulnérable (Acceptation du Risque)** :
  — Le serveur répond par un `ServerHello` et termine le handshake.
  — Résultat : `True` (Positif).
— **Scénario Sécurisé (Refus du Risque)** :
  — Le serveur coupe la connexion ou renvoie une **Alerte TLS**.
  — Résultat : `False` (Négatif).

11.3.3 Diagramme de séquence Tests de Configuration Forcée

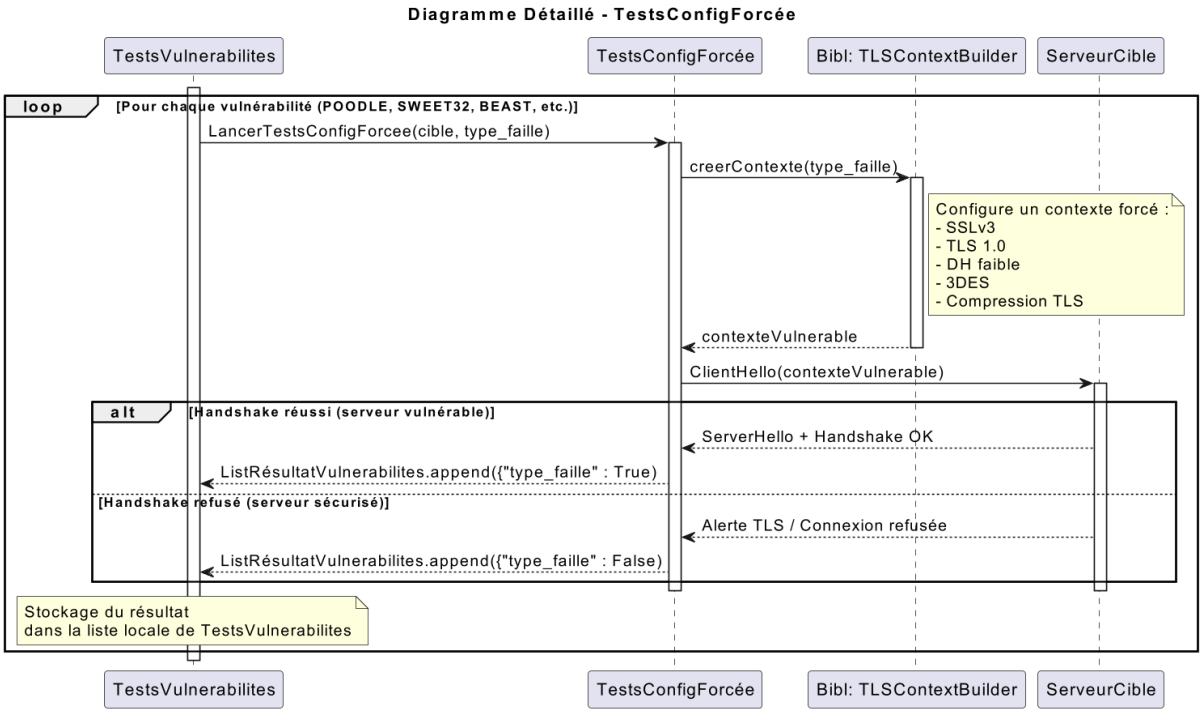


Diagramme de séquence Tests de Configuration Forcée

12 Fonctionnement Dynamique Test de Compatibilité OS

12.1 Liste des Composants Mis en Jeu

L'analyse de compatibilité repose sur l'interaction entre le moteur principal et un module de vérification croisée :

— **TestGlobal** : Composant principal du moteur d'analyse. Il pilote l'exécution séquentielle des différents tests (Certificat, Protocoles, Ciphers, etc.).

— **TestCompatibiliteOS** : Classe spécialisée chargée de vérifier la compatibilité théorique d'une liste de systèmes d'exploitation avec la configuration TLS/SSL détectée sur le serveur.

12.2 Description du Processus de Mise en Œuvre

Ce processus suit le flux d'événements du scénario "Vérifier la compatibilité avec les OS" et se déroule en trois phases.

12.2.1 Phase 1 : Récupération et Préparation des Données

— **Instanciation** : `TestGlobal` instancie un objet de la classe `TestsCompatibiliteOS`.
— **Appel** : `TestGlobal` demande a cet objet d'exectuer sa fonction `lancer()` avec en paramètre :
  — `listeResultats` : La liste consolidée contenant les résultats techniques déjà collectés sur le serveur (Certificat, Protocoles supportés, Ciphers supportés).
— **Chargement des Références** : La fonction charge en interne une liste de référence (`listeOs`).
  — Cette liste contient les capacités clients (Protocoles et Ciphers supportés) pour chaque OS (ex : Windows 10, iOS 16).

À Savoir !

Cette source de données (listeOS) est définie dans le code ou un fichier externe (JSON/YAML) et n'est pas transmise par le moteur principal.

12.2.2 Phase 2 : Algorithme de Vérification (Boucle de Compatibilité)

Le système itère sur chaque système d'exploitation présent dans la liste de référence `listeOs` :

1. **Extraction des Capacités** : Récupération de la configuration TLS/SSL supportée par l'OS cible (Protocoles + Ciphers).
2. **Comparaison Croisée** : Le système compare ces capacités avec la configuration réelle du serveur :
   — *Check Protocoles* : Vérifie s'il existe une intersection non vide entre les protocoles du serveur et ceux de l'OS.
   — *Check Ciphers* : Vérifie s'il existe au moins une suite de chiffrement commune.
   — *Check Certificat* : Vérifie les contraintes de signature si nécessaire.
3. **Verdict** : Détermination du statut booléen (`True` si compatible, `False` sinon).
4. **Enregistrement** : Mise à jour de la liste des résultats avec le statut et la raison éventuelle de l'incompatibilité.

12.2.3 Phase 3 : Sortie Finale (Structure de Données)

La fonction retourne une liste structurée d'objets JSON décrivant la compatibilité pour chaque OS testé.

```
[
  {
    "os": "Windows 7",
    "compatible": False,
    "raison": "TLS 1.2 requis non supporté par le serveur"
  },
  {
    "os": "Ubuntu 20.04",
    "compatible": True,
    "raison": ""
  }
  ...
]
```

12.2.4 Diagramme de séquence Compatibilité OS

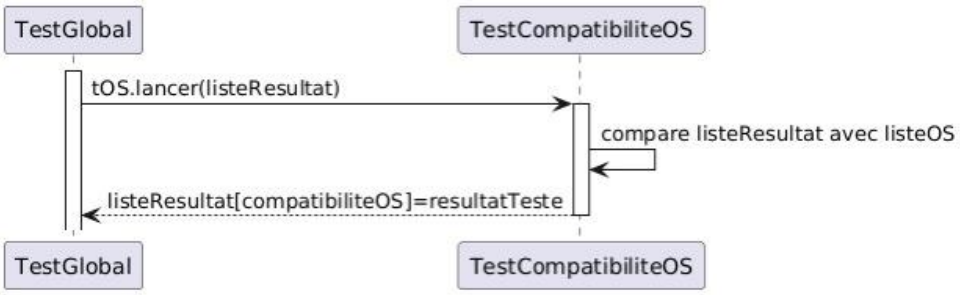


Diagramme de séquence Compatibilité OS

13 Fonctionnement Dynamique du Scan de Ports

13.1 Liste des Composants Mis en Jeu

Le processus d'analyse réseau s'appuie sur les acteurs suivants :

— **TestGlobal, Serveur Cible**.

— **TestScanDePorts (TestSP)** : Composant spécialisé responsable de l'analyse des ports TCP. Il génère les sondes réseau, interprète les réponses et classifie les ports (Ouverts, Filtrés, Inconnus).

13.2 Description du Processus de Mise en Œuvre

Ce processus suit le flux séquentiel du scénario d'analyse portuaire, divisé en trois phases.

13.2.1 Phase 1 : Lancement et Préparation

1. **Déclenchement** : `TestGlobal` active le module spécialisé en appelant la méthode :

   `lancer(FQDN)`

2. **Initialisation** : `TestSP` reçoit le FQDN, valide le format et prépare les structures de données (listes vides) et la configuration du scan.

13.2.2 Phase 2 : Phase Technique – Scan TCP SYN

Une fois l'IP obtenue, `TestsSP` itère sur une liste de ports prédéfinis (ex : plage 1–1024 ou liste critique).

Boucle d'Analyse (Par Port) Pour chaque port ciblé, la logique suivante est appliquée :

1. **Envoi de la Sonde** : `TestSP` envoie un paquet TCP avec le flag **SYN** (demande de synchronisation).
2. **Attente** : Le système attend une réponse avec un *timeout* défini (ex : 2 secondes).
3. **Classification de la Réponse** :
   — **Reçu [SYN/ACK] → Port Ouvert**
   Le serveur accepte la connexion.
   — **Reçu [RST] → Port Fermé**
   Le serveur refuse explicitement la connexion (généralement ignoré par l'outil).
   — **Aucune réponse (Timeout) → Port Filtré**
   Le paquet est probablement bloqué silencieusement par un pare-feu.

— **Comportement Ambigu → Port Inconnu**
Réception de paquets ICMP inattendus ou réponses incomplètes.

Stockage Interne Durant cette boucle, `TestsSP` remplit progressivement trois listes distinctes :

— `listePortsOuverts = []` : Appel de `ajouterPortOuvert(port)`
— `listePortsFiltres = []` : Appel de `ajouterPortFiltre(port)`
— `listePortsInconnus = []` : Appel de `ajouterPortInconnu(port)`

13.2.3 Phase 3 : Transmission des Résultats

Une fois l'ensemble des ports analysés :

1. `TestsSP` consolide les informations collectées dans une structure finale contenant les trois listes (Ouverts, Filtrés, Inconnus).
2. Il renvoie cet objet de résultat à `TestGlobal` pour intégration dans le rapport de sécurité.

13.2.4 Diagramme de Séquence Scan de Ports

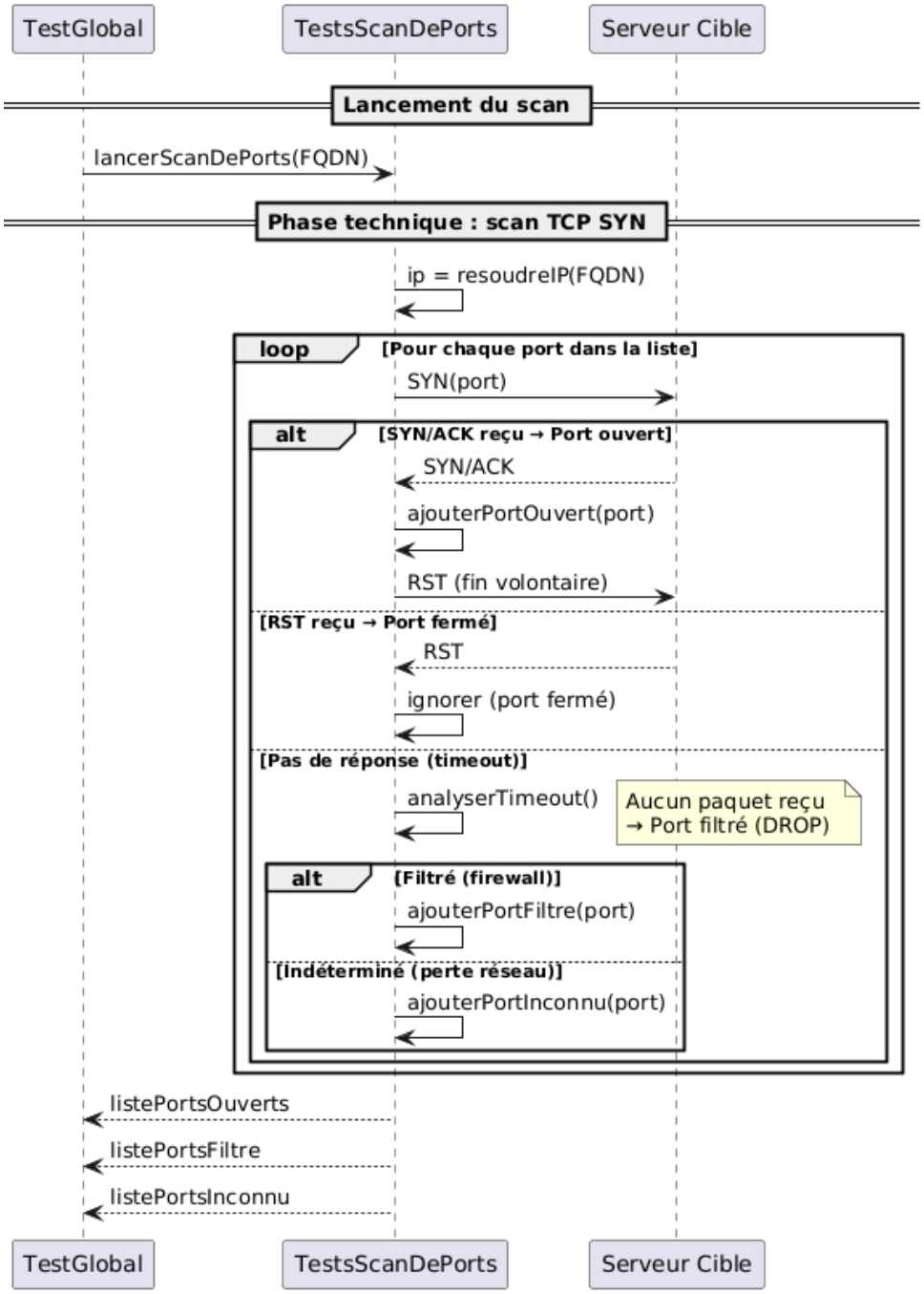


Diagramme de séquence Compatibilité OS

14 Note et Rapport

14.1 Calcul de la Note Globale de Sécurité

La notation est radicale et simple. Nous avons voulu être sévères sur l'évaluation : être sécurisé, c'est **To Be or Not To Be**.

14.1.1 Règle de Conditionnement Initiale (Certificat)

Cette règle impose une note de 0 pour tout défaut fondamental sur le certificat.

Condition (Certificat)	**Conséquence**
Certificat invalide, expiré, auto-signé, non-trusted ou révoqué	**Note Finale = T**
Certificat valide et trusted	On continue le scoring pondéré

14.1.2 Règles de Scoring Pondéré par Catégorie Technique

Si le certificat est valide, la note globale est calculée sur la base des règles suivantes :

Critère 1 : Protocoles Supportés (Poids : 30%) La note est calculée selon une méthode cumulative :

1. On identifie le meilleur protocole supporté (ordre : TLS 1.3 > 1.2 > 1.1 > 1.0)
2. On prend sa note de base
3. On applique les pénalités pour les autres protocoles supportés

Protocole	**Note si meilleur**	**Pénalité**
`TLS 1.3`	100	-0
`TLS 1.2`	80	-5
`TLS 1.1`	50	-10
`TLS 1.0`	30	-20
`SSL 2.0/3.0`	Note = 0 (bloquant)	

Exemple de calcul :

— TLS 1.3 seul : 100
— TLS 1.3 + TLS 1.2 : 100 - 5 = 95
— TLS 1.2 seul : 80
— TLS 1.2 + TLS 1.1 : 80 - 10 = 70
— TLS 1.2 + TLS 1.0 : 80 - 20 = 60

Critère 2 : Robustesse de la Clé (Key Exchange) (Poids : 30%) La note est calculée par moyenne sur l'ensemble des mécanismes d'échange de clés supportés.

Règle bloquante :

— Présence de `ANON`, `EXPORT` ou `NULL` → Note = 0

Notation des courbes ECDHE :

Taille de la courbe	**Note**
Taille $\geq$ 256 bits	100
Taille $<$ 256 bits	0

Notation des clés RSA, DHE, DH :

Taille de la clé	Note
≥ 7680 bits	100
≥ 4096 bits	90
≥ 3072 bits	80
≥ 2048 bits	70
≥ 1536 bits	40
≥ 1024 bits	20
< 1024 bits	0

La note finale du critère est la moyenne des notes de tous les mécanismes supportés.

Critère 3 : Force du Chiffrement (Cipher Strength) (Poids : 40%) La note est calculée par moyenne sur l'ensemble des cipher suites supportées. Chaque cipher reçoit une note individuelle selon les règles suivantes :

Type de Cipher	Note de base
`NULL`, `ANON`, `EXPORT`, `MD5`, `DES`	0
`RC4`	20
`3DES`	40
`CBC`	60
`GCM`, `CHACHA20`, autres	100

Taille de clé	Ajustement
Taille < 128 bits	Note $= \min(\text{note_base}, 40)$
Taille ≥ 256 bits	Note $= \min(\text{note_base} + 10, 100)$

La note finale du critère est la moyenne des notes individuelles de tous les ciphers supportés.

Critère 4 : Certificat (Note indépendante) Le certificat est noté sur 100 points selon la grille suivante :

Critère	Points max
Validité temporelle	25
Chaîne de certification	25
Force de la clé publique	20
Algorithme de signature	10
Extensions de sécurité	10
Statut de révocation	10

Détail des notations et pénalités :

Validité temporelle (25 pts) :

— Certificat expiré ou pas encore valide → 0 pt

— Expire dans moins de 30 jours → 15 pts

— Sinon → 25 pts

Chaîne de certification (25 pts) :

— Chaîne incomplète → -10 pts

— Reconstruite via AIA → -10 pts

— Longueur > 5 certificats → -5 pts

Force de la clé publique (20 pts) :

— RSA ≥ 2048 bits ou EC ≥ 256 bits → 20 pts

— EC < 256 bits → 10 pts

— Sinon → 0 pt

Algorithme de signature (10 pts) :

— Hash faible (MD5, SHA1) → 0 pt

— Hash sécurisé (SHA256, SHA384, SHA512) → 10 pts

Extensions de sécurité (10 pts) :

— Pas de SAN → -2 pts

— Pas de Key Usage → -2 pts

— Pas d'OCSP dans AIA → -5 pts

— Pas de caIssuers dans AIA → -2 pts

Statut de révocation (10 pts) :

— Aucun mécanisme → 0 pt

— CRL uniquement → 5 pts

— OCSP uniquement → 8 pts

— OCSP et CRL disponibles → 10 pts

Pénalités supplémentaires :

— Certificat auto-signé → -50 pts

— CN ne correspond pas au FQDN → -30 pts

La note finale du certificat est plafonnée entre 0 et 100.

14.1.3 Formule de Calcul et Conversion du Grade

La note finale est le résultat de la pondération, convertie selon une échelle stricte :

$$\text{Note Finale} = \begin{cases} 0 \text{ si certificat_invalide} \\ (P \times 0.30) + (K \times 0.30) + (C \times 0.40) \quad \text{sinon} \end{cases}$$

Note Numérique	Grade Lettre	Interprétation
-	**T**	**Trusted Issue** (Certificat invalide)
0 – 49	**F**	**Fail** (Non sécurisé)
50 – 70	**B**	**B** (Bon / À améliorer)
71 – 100	**A**	**A** (Sécurisé / Excellent)

Scénarios de Notation (Simulation des cas possibles) Ce tableau illustre comment la note bascule d'un grade à l'autre selon les configurations.

Configuration Type	P	K	C	Note	Grade
1. Parfaite (TLS 1.3 seul)	100	100	100	**100**	**A**
2. Standard (TLS 1.3 + TLS 1.2, RSA 2048, AES128)	95	80	80	**84,5**	**A**
3. TLS 1.2 uniquement	80	80	80	**80**	**A**
4. TLS 1.2 + TLS 1.0 (protocole faible)	60	100	100	**82**	**A**
5. Clé Faible (RSA 1024)	80	0	80	**56**	**B**
6. Passoire Crypto (Clé et Cipher faibles)	80	0	0	**24**	**F**
7. Certificat Expiré	-	-	-	**-**	**T**

14.2 Structuration du Résultat Final

La consolidation de la réponse finale est une responsabilité de l'orchestrateur `SSLChecker` et non des modules `NoteGlobal` et `RapportGlobal`, qui travaillent indépendamment.

14.2.1 Rôles des Composants dans la Restitution

1. **NoteGlobal** : Calcule et stocke la **Note Finale** et toutes les **sous-notes**.
2. **RapportGlobal** : Agrège et structure tous les **sous-rapports** bruts des tests techniques.
3. **SslChecker** : Est le seul point qui demande les notes à `NoteGlobal` et les rapports à `RapportGlobal`. Il transmet ces deux objets au Contrôleur.
4. **Contrôleur** : Reçoit les deux structures et les sérialise dans la réponse finale (JSON).

14.2.2 Structure de l'Objet de Sortie JSON

La réponse finale du Contrôleur fusionne les deux objets collectés par `SSLChecker` sous la forme d'une structure simple et claire :

```
reponse = {
    "note": {
        "note_globale": 88,
        "grade": "A",
        "note_protocoles": 80,
        "note_cle": 80,
        ...
    },
    "rapport": {
        "rapport_certificat": {
            "validite": "OK",
            "issuer": "Let's Encrypt"
        },
        "rapport_protocoles": {
            "tls1_3": true,
            "ssl3_0": false
        },
        ...
    }
}
```

15 Fonctionnement du Frontend

15.1 Liste des Composants de l'Interface Utilisateur

L'interface utilisateur est faite pour être légère et ergonomique , reposant sur une stack standard sans framework lourd, elle sera entièrement développer en HTML, CSS et JS.

Zone de Saisie Formulaire permettant à l'utilisateur de définir la cible. Il contient :
- Un champ texte pour le **FQDN** (ex : `admin.tryhackme.com`).
- Un champ numérique optionnel pour le **Port** (Par défaut : `443`).
- Un bouton d'action **"Lancer l'Analyse"** qui déclenche le processus.

Code JavaScript Le script principal qui gère la logique asynchrone. Il intercepte l'action de l'utilisateur, communique avec le **Contrôleur** via l'API Fetch. Il manipule le DOM pour afficher les résultats et la progression des tests en temps réel.

Conteneur de Résultats) Une zone initialement masquée qui se divise en plusieurs blocs (Score, Certificat, Protocoles...) destinés à recevoir les données une fois l'analyse terminée.

15.2 Description du Processus de Mise en Œuvre

Le flux d'exécution côté client se déroule en trois phases, depuis l'interaction utilisateur jusqu'à l'affichage des résultats.

15.2.1 Phase 1 : Saisie et Requête Asynchrone

1. **Action Utilisateur** : L'utilisateur entre le FQDN[port] et clique sur le bouton.
2. **Validation Préalable** : Le JavaScript vérifie que le champ FQDN n'est pas vide. Si le port est omis, il force la valeur par défaut `443`.
3. **Appel API** :
   — Il envoie une requête `GET` vers le **Contrôleur** avec les paramètres cibles.
   — Il affiche la progression des tests en temps réel pour l'utilisateur.

15.2.2 Phase 2 : Réception de la Réponse

Une fois l'analyse terminée, le **Contrôleur** renvoie l'objet JSON (voir structure définie précédemment en 12.2.2).

1. **Réception** : JS reçoit l'objet `reponse` contenant les deux champs principaux : `note` et `rapport`.
2. **Parsing** : Le script sépare les données pour les diriger vers les fonctions d'affichage dédiées.

15.2.3 Phase 3 : Affichage Des Blocs

L'interface se construit bloc par bloc en injectant le HTML et CSS nécessaire.

16 Traçabilité

Cette section établit la correspondance entre les exigences de la Spécification Technique des Besoins (STB) et les composants logiciels développés dans ce Dossier d'Architecture Logicielle (DAL).Elle prouve que la conception couvre l'intégralité des besoins fonctionnels et non fonctionnels majeurs.

16.1 Fonctionnelles

ID Exigence	**Description simplifiée (Règle de Gestion)**	**Composant impliqué**
EF-01	Saisie de l'adresse cible (FQDN [Port]).	Frontend / Contrôleur / SslChecker
EF-02	Vérification de la connectivité.	SslChecker
EF-03	Récupération du certificat serveur.	TestCertificat
EF-04	Vérification de l'expiration du certificat.	TestCertificat
EF-05	Vérification de l'émetteur du certificat.	TestCertificat
EF-06	Analyser la validité de la chaîne de certificats.	TestCertificat
EF-07	Détection des protocoles acceptés.	TestProtocole
EF-08	Détection des protocoles obsolètes.	RapportProtocole
EF-09	Détection des suites de chiffrement du serveur.	TestCipher
EF-10	Identifier les cipher suites obsolètes ou vulnérables.	RapportCipher
EF-11/12	Tester pour les vulnérabilités majeures.	TestsVulnerabilites
EF-13	Lister tous les OS compatibles avec le serveur.	TestCompatibiliteOS
EF-14/15	Effectuer un scan des ports principaux.	TestScanDePorts

Suite à la page suivante...

ID Exigence	Description simplifiée (Règle de Gestion)	Composant impliqué
EF-16	Attribuer une note pour la qualité du certificat.	NoteCertificat
EF-17	Attribuer une note pour la configuration des protocoles.	NoteProtocole
EF-18	Attribuer une note pour la configuration des cipher suites.	NoteCipher
EF-19	Calculer une note globale.	NoteGlobale / NoteX
EF-20	Affichage d'un rapport complet et lisible à l'utilisateur.	Frontend / RapportGlobal / RapportX
EF-21	Vérifier la préférence du serveur pour l'ordre des ciphers.	TestCipher
EF-22	Vérifier les extensions (SAN, EKU) du certificat.	TestCertificat
EF-23	Vérifier le statut de révocation (OCSP/CRL).	TestCertificat
EF-24	Analyser la robustesse de la clé publique et signature (RSA/SHA).	TestCertificat
EF-25	Déterminer les algorithmes d'échange de clés supportés.	TestEchangeCle
EF-26	Déterminer la taille de la clé d'échange associée.	TestEchangeCle

16.2 Non-Fonctionnnelles

ID Exigence	Description	Composant impliqué
ENF-01	Temps d'analyse raisonnable (< 5 min) et interface réactive.	SSLChecker
ENF-02	Accessibilité de l'outil en intranet.	Environnement Docker / Nginx/Gunicorn
ENF-03	Interface intuitive et claire.	Frontend
ENF-04	Gestion robuste des erreurs.	L'ensemble des composants
ENF-05	Protection contre les injections.	Contrôleur / Nginx / SslChecker
ENF-06	Exactitude des résultats et des détections.	TestGlobale
ENF-07	Code structuré pour faciliter les ajouts (Architecture modulaire).	Backend (code modulaire)
ENF-08	Compatibilité avec environnements standards (Support Linux/Windows).	Environnement Docker / Python/Flask